Department of Statistics 2024–25

Portfolio Optimisation via Mean Field Games with Common Noise using Signatured Deep Fictitious Play, and Learning Rate Tuned Neural Networks

Candidate Number:44369

Submitted for the Master of Science,
London School of Economics, University of London

Contents

Summary v

1 Introduction 2
- 1.0.1 Our Contributions . . . 2
- 1.0.2 Key Symbols and Notation . . . 3

2 Background and Literature Review 5

3 Learning MFGs with Common Noise via Signatured Deep Fictitious Play 12
- 3.1 Introduction to Rough Path Theory and Signatures . . . 12
- 3.2 Literature Review . . . 15
  - 3.2.1 Traditional PDE Methods for MFGs . . . 15
  - 3.2.2 Fictitious Play and Extensions to Common Noise . . . 15
  - 3.2.3 Signatured Deep Fictitious Play (Min & Hu, 2021) . . . 15
- 3.3 Signatured Deep Fictitious Play (Sig-DFP) Algorithm . . . 16

4 Problem statement (MFG with common noise) 17
- 4.1 Computational Efficiency and Convergence Analysis . . . 18
- 4.2 Computational Procedure . . . 18
  - 4.2.1 Contributions of this Dissertation . . . 19

5 Methodology 21
- 5.1 Stage I: Stabilizing Convergence in Sig-DFP . . . 21
  - 5.1.1 Stabilized Sig-DFP Backbone . . . 21
- 5.2 Stage I: Fixing the conditional–law step in Sig-DFP . . . 22
  - 5.2.1 Backbone (common across all three fixes) . . . 22
  - 5.2.2 Approach A: Generative push–forward on signatures . . . 23
  - 5.2.3 Approach B: Path–to–value regression (direct conditional mean) 23
  - 5.2.4 Approach C: Kernel regression on signatures (mixture estimator) 24
  - 5.2.5 Diagnostics and tuning (common) . . . 24

6 Stage I: Fixing the Mean-Field Lemma 25
- 6.1 Stage II: Portfolio Optimisation Extension . . . 30
- 6.2 Benchmarks and risk metrics . . . 30
- 6.3 Policy architectures and learner . . . 31

7 Stage II Results: Portfolio Optimisation Experiments 33
- 7.1 Scope and assumptions . . . 33
- 7.2 Detailed analysis of results . . . 33

8 Conclusions **37**

8.1 Future Work: Dynamic Covariance Models for Common Shocks 38

Bibliography **39**

9 Code Repository **42**

List of Figures

1 (A) Baseline (no fix): oscillatory loss; large residuals. 25
2 (B) Path-to-Value regressor: fast contraction. 26
3 (C) Pushforward (GAN-style): stable, slightly higher plateau. 26
4 (D) Kernel regression (auto-γ): strongest contraction. 27
5 **Sharpe by architecture and tuning (mean across runs, fast1000).** 33
6 **Equity curves (top Sharpe run per arch/tune, fast1000).** 34
7 **Drawdowns (top Sharpe run per arch/tune, fast1000).** 35
8 **Rolling Sharpe (126-day window, top runs, fast1000).** 36

List of Tables

1 Stage I comparison of lemma fixing strategies under the STRESS regime (LSTM policies). Lower MF-gap and exploitability are better. 28
2 Leaderboard (top runs by Sharpe, fast1000). Includes financial and error metrics. 34
3 Validated runs used in analysis (top Sharpe per arch/tune). Annualised metrics. 35
4 Bootstrap differences in Sharpe (bo – base), with 95% confidence intervals. 36

Summary

We study portfolio optimisation in mean field games with common noise, proposing a single-loop Signatured Deep Fictitious Play solver with stability fixes (graph detachment and EMA mean-field updates) and a portfolio-aware loss. We clarify convergence assumptions for learning in common-noise MFGs, and empirically compare conditional-law approximators (path-to-value, pushforward, kernel regression) using signature features of the common noise. Kernel regression delivers the most contractive fixed-point behaviour and is adopted in Stage II portfolio experiments. Across feed-forward, LSTM, and Transformer policies, the feed-forward network paired with signatures achieves the strongest risk-adjusted performance at modest compute, while LSTMs minimise drawdowns. We outline an extension that conditions on dynamic covariance (DCC–GARCH) to better capture clustered volatility and correlation spikes.

Acknowledgements

First and foremost, I would like to thank my parents for their constant support and encouragement. Their faith in me has been a steady source of motivation throughout this project. I am also deeply grateful to my supervisor, Dr. Giulia Livieri. Her guidance, deep expertise, and positive energy have been invaluable at every stage. This dissertation quite simply would not have taken shape without her advice and encouragement.

1 Introduction

In practice, investors are always reacting to two different kinds of information. On one hand there are shocks that are specific to them as individuals (idiosyncratic shocks), and on the other there are big market-wide events that affect everyone at once (what we will call "common noise"). When a very large number of investors respond simultaneously, their actions feed back into the market as a whole. The outcome is a complex and constantly shifting feedback loop.

Mean Field Games (MFGs) provide a natural mathematical framework for modelling this kind of setting. Instead of tracking how every single investor interacts with every other one, an MFG assumes that each agent reacts to the average behaviour of the population. This simplification works very well in large systems. However, once common noise is introduced, the standard MFG machinery runs into two major problems:

1. The mean field is no longer a simple, predictable path. It becomes a random process, driven entirely by the particular sequence of common shocks that have happened. This adds a lot of complexity.

2. Straightforward learning algorithms, where investors try to adapt their strategy based on the observed crowd, tend to be fragile. They can loop back on themselves or diverge, failing to settle on a sensible equilibrium.

Problem. The central challenge of this dissertation is to design a method that can compute a stable equilibrium strategy for portfolio management in the presence of common noise. The method needs to be efficient and scalable (a "single-loop" algorithm), and its outputs—such as utility, Sharpe ratio, and drawdown—should be directly relevant and interpretable for practitioners like asset managers.

1.0.1 Our Contributions

To address this, the dissertation makes four main contributions:

1. **A More Stable Learning Algorithm.** We implement Signatured Deep Fictitious Play (Sig–DFP) and introduce two critical modifications: computational-graph detachment and exponentially smoothed updates. Together these resolve the instability problems that affect naive versions and make the learning dynamics far more reliable.

2. **A Financially-Minded Loss Function.** We design a portfolio-aware loss function that directly incorporates the mean–variance trade-off. This serves two purposes: it makes the optimisation more interpretable in financial terms, and it helps to further stabilise training.

3. **A Corrected Theoretical Foundation.** We revisit and refine a key mathematical proof from the literature. By clarifying the assumptions and correcting the argument, we place the method on a more solid theoretical footing within established MFG theory.

4. **A Comprehensive Empirical Study.** We carry out a large-scale experiment across six different neural architectures, testing both normal and stressed market regimes. We also explore Bayesian optimisation for hyperparameters. The end result is a detailed, comparative view of what actually works in practice.

1.0.2 Key Symbols and Notation

Before we proceed, here is a guide to the main symbols that will appear throughout.

- $t \in [0, T]$: Continuous **time**, measured in years. Often discretised on a grid t_k.
- X_t: An investor's **wealth** at time t, starting from $x_0 > 0$.
- $\pi_t \in [0, 1]$: The investor's **portfolio policy**—the fraction of wealth allocated to the risky asset at time t. This is the control variable of interest.
- W_t: **Idiosyncratic noise**, unique to each investor, modelled as a standard Brownian motion.
- B_t: **Common noise**, representing market-wide shocks, also a Brownian motion and independent of W_t.
- $m_t := E[X_t|\mathcal{F}_t^B]$: The **mean field**, i.e. the conditional average wealth across all investors given the history of the common shocks.

The wealth dynamics are written as a stochastic differential equation (SDE). In general:

$$X_{t_{k+1}} = X_{t_k} + b(t_k, X_{t_k}, m_{t_k}, \pi_{t_k})\Delta t \ + \ \sigma(\cdot)\Delta W_{t_k} \ + \ \sigma_0(\cdot)\Delta B_{t_k}.$$

For our portfolio setting this reduces to a familiar geometric Brownian motion (GBM):

$$dX_t = \pi_t X_t\big(\mu\, dt + \nu\, dW_t + \sigma_c\, dB_t\big),$$

which corresponds to

$$\begin{cases} b(t, x, m, \pi) = \pi\mu x, \\ \tilde{\sigma}(t, x, m, \pi) = \pi\nu x, \\ \sigma_0(t, x, m, \pi) = \pi\sigma_c x. \end{cases}$$

Finally,

- $S_M(\hat{B}_{0:t})$: the **path signature** of the common noise up to t, truncated at depth M. These features encode path history in a way that is both compact and expressive for learning.

2 Background and Literature Review

Before turning to the specific contributions, it is important to establish the theoretical foundations of this work. We begin with classical game theory and its extension to Mean Field Games (MFGs), and then review the tools from stochastic calculus and learning theory that underpin our methodology.

Game Theory and Nash Equilibrium

At its heart, game theory is about studying how rational decision-makers—called agents or players—interact when their choices affect one another. The field was formally introduced by John von Neumann and Oskar Morgenstern in their classic 1944 book, *Theory of Games and Economic Behavior* von Neumann and Morgenstern (1944), which laid the groundwork for modelling strategic and economic behaviour. Since then, game theory has spread far beyond economics; it is now used in biology (to model evolution and cooperation), in computer science (for algorithms and networks), and in many other areas where strategic decision-making plays a role.

A central idea in this field is the **Nash equilibrium**, named after John Nash who introduced it in 1950Nash (1950). A Nash equilibrium is a stable outcome of a game where each player's strategy is the best possible response to the strategies of everyone else. Put differently: once the system reaches a Nash equilibrium, no individual player can unilaterally change their strategy and end up strictly better off, as long as all the other players keep theirs fixed.

Formally, consider a finite set of players $I = \{1, \dots, n\}$. Each player $i \in I$ chooses an action from their strategy set $\mathcal{A}_i$, aiming to maximise their payoff function $f_i : \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \mathbb{R}$(Fudenberg and Levine, 1998). A joint action of all players is denoted by $a = (a_1, \dots, a_n)$, while the profile of everyone except player i is written as $a_{-i} = (a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$.

A profile $a^* = (a_1^*, \dots, a_n^*)$ is a Nash equilibrium if, for every player i, the strategy a_i^* is an optimal response to a_{-i}^*:

$$f_i(a_i^*, a_{-i}^*) \;\geq\; f_i(a_i, a_{-i}^*) \quad \forall a_i \in \mathcal{A}_i,\ \forall i \in I. \qquad \text{(Nash, 1950)}$$

(We will later reserve upper-case X for wealth; using a for actions avoids notational clashes.)

MFGs with Common Noise

Classical game theory is well-suited for a handful of players, but becomes unwieldy when we try to describe the behaviour of large populations (such as in financial markets).

This is where *Mean Field Games (MFGs)* come in. First developed by Lasry and Lions Lasry and Lions (2007), and independently by Huang, Malhamé, and Caines, MFGs formalise what happens when the number of players grows very large.

The key idea is the *mean-field approximation*: instead of tracking every pairwise interaction, each agent interacts only with the statistical average of the population (the "mean field"). An equilibrium in this framework is characterised by two coupled PDEsGuéant et al. (2011); Lasry and Lions (2007):

1. The **Hamilton–Jacobi–Bellman (HJB) equation**, which describes the optimal control problem of a representative agent facing a given mean-field evolution.

2. The **Fokker–Planck (Kolmogorov Forward) equation**, which tracks the time evolution of the population distribution when all agents follow their optimal strategies.

The system is naturally forward–backward: the HJB must be solved backwards in time (optimal controls depend on future states), while the Fokker–Planck flows forward in time (population dynamics given controls).

From Physics to MFGs

The analogy comes from physics. In large particle systems, mean-field theory replaces pairwise forces with an averaged interaction, drastically reducing complexity. In socio-economic settings, we do something similar, but with one twist: agents are not passive like particles. Instead, they actively optimise their behaviour, anticipating the reactions of the "crowd."

In our financial context, each agent's wealth X_t evolves according to a multiplicative SDE (Merton-style)(Merton, 1969, 1971):

$$\mathrm{d}X_t \;=\; \pi_t\, X_t\big(\mu\,\mathrm{d}t + \nu\,\mathrm{d}W_t + \sigma\,\mathrm{d}B_t\big), \qquad X_0 > 0, \tag{1}$$

where π_t is the portfolio weight in the risky asset, μ its drift, ν the idiosyncratic volatility (driven by W_t), and σ the common volatility (driven by B_t). Here W is idiosyncratic Brownian motion and B is the common Brownian motion.

Mapping of coefficients (general → portfolio). For the portfolio case,

$$\mathrm{d}X_t = \pi_t X_t\big(\mu\,\mathrm{d}t + \nu\,\mathrm{d}W_t + \sigma_c\,\mathrm{d}B_t\big),$$

the McKean–Vlasov coefficients map as

$$b(t,x,m,\pi) = \pi\mu x, \qquad \tilde{\sigma}(t,x,m,\pi) = \pi\nu x, \qquad \sigma_0(t,x,m,\pi) = \pi\sigma_c x,$$

with W idiosyncratic and B common. We use σ_c specifically for the common-noise scale (to avoid confusion with σ as a generic diffusion term).

Solving the agent's optimal control problem in the mean-field limit then leads to a forward–backward PDE system: the HJB (for individual optimality) and the Fokker–Planck (for the collective distribution)Guéant et al. (2011); Lasry and Lions (2007).

Mathematical Formulation of MFGs

Formally, fix a mean-field flow $m = (m_t)_{0 \le t \le T}$. The representative agent minimises

$$J(\pi; m) := E\left[\int_0^T \ell(t, X_t, \pi_t, m_t)\, dt + g(X_T, m_T) \;\middle|\; X_0 = x_0\right],$$

with value function $V(t, x; m) = \inf_\pi E_{t,x}[\cdots]$. The HJB equation is:

$$\partial_t V(t, x; m) + \inf_{\pi \in \mathcal{A}} \left\{ \pi \mu x\, V_x + \tfrac{1}{2}\pi^2(\nu^2 + \sigma_c^2)x^2 V_{xx} + \ell(t, x, \pi, m) \right\} = 0,$$
$$V(T, x; m) = g(x, m_T). \quad (2)$$

In our portfolio setting:

$$\ell(t, x, \pi, m) = \eta\, \pi^2 + \beta\, (x - m)^2, \quad g(x, m) = -\log x, \quad \eta, \beta \ge 0.$$

We do not solve (2) directly; instead, we learn a feedback π_θ by minimising a discrete loss.

The companion Fokker–Planck equation is(Carmona and Delarue, 2018; Lasry and Lions, 2007, Chs. 2–3):

$$\frac{\partial m_t}{\partial t}(x) + \frac{\partial}{\partial x}[b(x, u^*(t, x))m_t(x)] = \tfrac{1}{2}\frac{\partial^2}{\partial x^2}[\sigma^2(x, u^*(t, x))m_t(x)], \quad (3)$$

with initial distribution $m_0(x)$.

Incorporating Common Noise

To capture macro shocks or policy moves that affect all agents at once, MFGs with *common noise* extend this framework. Now the mean-field distribution becomes stochastic itself, conditioned on the realised path of the common noise. Mathematically, this makes the population dynamics random rather than deterministic, and the PDEs become stochastic PDEsGuéant et al. (2011); Carmona et al. (2016).

Computational Challenges and Numerical Solutions

This extra stochasticity comes at a computational cost: one must now approximate conditional distributions across many realisations, usually via Monte Carlo or stochastic PDE solvers. The dimensionality quickly explodes. Neural-network-based methods have recently shown promise in handling this complexityMin and Hu (2021).

A particularly effective tool is the use of *signature features*, $S_M(\widehat{B}_{0:t})$Lyons et al. (2007), which encode the history of a stochastic path into a finite set of tensors. Truncating the signature at depth M yields a compact feature vector that retains rich path information. This dramatically reduces the cost of representing common noise in MFGs.

Applications in Finance and Asset Management

In finance, MFGs with common noise are especially relevant. They model competition among asset managers whose performance is measured relative to peers. Strategic interactions arise because each manager adjusts risk exposure not only based on market conditions but also on their ranking compared to others. Common noise captures systemic shocks, producing a richer and more realistic model of financial marketsGuéant et al. (2011). Applications include systemic risk, herding, liquidity crises, and price impact studiesMin and Hu (2021).

Challenges and Future Directions

Even so, major difficulties remain:

- computational complexity from stochastic distributions and high-dimensional PDEs,
- ensuring numerical convergence under common noise,
- designing advanced feature transforms (such as signatures) to handle path dependence, and
- establishing theoretical guarantees of equilibrium existence and uniqueness.

Despite these challenges, research is advancing quickly. New tools are extending both the analytical and numerical tractability of MFGs, making them increasingly central to modern economics and financeGuéant et al. (2011); Carmona et al. (2016).

Stochastic Calculus and Itô's Lemma

Stochastic calculus extends the familiar rules of ordinary calculus into settings where randomness is unavoidable. The point is to analyse systems that evolve under continuous-time noise, typically represented by Brownian motion, and to do so rigorously. This is indispensable in finance (modelling asset prices), physics (particle dynamics), engineering (signal noise), and many other areas. The central aim is to provide the tools to write down and solve differential equations where the drivers are random processes rather than smooth functions.

Historical Development

The subject did not emerge overnight but was built step by step.

- **Norbert Wiener (1923)** gave the first rigorous definition of Brownian motion—the Wiener process—formalising the irregular trajectories observed experimentally by Robert Brown a century earlier Wiener (1923).
- **Andrey Kolmogorov (1931)** then established the axiomatic basis for probability theory, providing the machinery to handle stochastic processes consistently Kolmogorov (1931).
- **Kiyoshi Itô (1944)** pushed this further, introducing stochastic integrals and formulating what we now call **Itô's Lemma**—arguably the central result of the whole subject, since it extends the chain rule to stochastic processes Itô (1944).

Mathematical Formulation of Itô's Lemma

For presentation, we use geometric Brownian motion (GBM) as the canonical model of portfolio dynamics. The learning pipeline and mean-field update would be the same for any other Itô diffusion (Merton, 1969, 1971; Shreve, 2004).

$$dX_t = \mu X_t \, dt + \sigma X_t \, dW_t, \tag{4}$$

where:

- μ is the *drift coefficient* (the deterministic trend or expected growth),
- σ is the *volatility coefficient* (capturing the random fluctuations),
- W_t is standard Brownian motion (a Wiener process, with Gaussian increments) (Wiener, 1923; Kolmogorov, 1931; Shreve, 2004).

Now suppose $f(t, X_t)$ is a smooth enough function (twice differentiable in x, once in t). Then Itô's Lemma Itô (1944); Øksendal (2003); Shreve (2004) tells us:

$$df(t, X_t) = \frac{\partial f}{\partial t}\, dt + \frac{\partial f}{\partial x}\, dX_t + \tfrac{1}{2}\frac{\partial^2 f}{\partial x^2}\, \sigma^2 X_t^2\, dt. \tag{5}$$

This splits naturally into:

- **Deterministic contributions** (all terms proportional to dt):

$$\frac{\partial f}{\partial t}\, dt \;+\; \frac{\partial f}{\partial x}(\mu X_t\, dt) \;+\; \tfrac{1}{2}\frac{\partial^2 f}{\partial x^2}\, \sigma^2 X_t^2\, dt.$$

- **Stochastic contribution** (the part driven by noise):

$$\frac{\partial f}{\partial x}\, (\sigma X_t\, dW_t).$$

The subtlety is that $\frac{\partial f}{\partial x} dX_t$ contains both a drift piece and a diffusion piece. Only the latter carries genuine randomness; the drift folds into the deterministic terms. The derivation hinges on a key difference from classical calculus: for Brownian motion $(dW_t)^2 = dt$ (whereas $(dx)^2$ would vanish in ordinary calculus). This single fact underpins much of stochastic analysis Shreve (2004).

Applications of Itô's Lemma

Itô's Lemma turns up everywhere randomness and optimisation meet:

- **Finance**: central to the Black–Scholes–Merton option pricing framework Black and Scholes (1973), and to modern risk management more broadly.
- **Stochastic control**: essential for portfolio optimisation, risk-sensitive control, and decision-making under uncertainty.
- **Mean Field Games**: used to set up the forward–backward PDE systems that define equilibria under noise Guéant et al. (2011); Min and Hu (2021).

In short: stochastic calculus, and Itô's Lemma in particular, provides the language and rules for working rigorously with random dynamical systems.

Fictitious Play: Development, Use, and Solution Process

Fictitious play is one of the simplest yet most influential learning rules in game theory. Agents repeatedly play a game, each time adjusting their strategy based on what others have done before, as if forecasting opponents' future behaviour from their past frequencies. Over time, these iterative updates can converge to a Nash equilibrium.

Historical Development

George W. Brown introduced fictitious play in 1951 Brown (1951). The original motivation was straightforward: compute equilibria by having each player best-respond to the empirical distribution of opponents' past strategies. Since then, it has been used in:

- **Economics**, for dynamic competition and bargaining Fudenberg and Levine (1998),
- **Reinforcement learning**, as a simple multi-agent learning rule,
- **Mean Field Games**, as a natural way to approximate equilibria when populations are large Cardaliaguet and Hadikhanloo (2017); Min and Hu (2021).

Mathematical Formulation

Formally, suppose player i chooses action a_t^i at time t. Then under fictitious play:

$$a_t^i = \arg\max_{a^i \in A_i} \sum_{s=1}^{t-1} f_i(a^i, a_s^{-i}), \tag{6}$$

where A_i is the strategy set for player i, f_i is their payoff, and a_s^{-i} denotes the actions of everyone else at time s. Each player is effectively maximising against the empirical distribution of others' past play.

Solution Process

The procedure runs as follows:

1. **Initialisation**: players pick arbitrary starting strategies.
2. **Observation**: they record the actions of others.
3. **Best response**: they update to their payoff-maximising action against the empirical frequencies.
4. **Iteration**: repeat the cycle.
5. **Convergence**: strategies settle down towards a Nash equilibrium Fudenberg and Levine (1998); Cardaliaguet and Hadikhanloo (2017).

This simple loop makes fictitious play a practical tool for approximating equilibria in repeated and large-population games.

3 Learning MFGs with Common Noise via Signatured Deep Fictitious Play

3.1 Introduction to Rough Path Theory and Signatures

Rough path theory, developed by Lyons in the late 1990s Lyons (1998), tackles a fundamental issue: many stochastic processes, especially in finance, produce sample paths that are far too irregular for standard calculus. Price trajectories can jump, spike, or cluster in volatility, and classical Itô calculus, while powerful, is not always robust to such irregularities (Friz and Victoir, 2010). Rough path theory extends the toolkit by providing stable analytic methods for non-smooth paths.

At the core is the **signature transform**. For a path X_t, its signature over $[0, T]$ is the sequence of iterated integrals:

$$S(X)_{0,T} = \left(1, X^1_{0,T}, X^2_{0,T}, \dots\right), \tag{7}$$

with

$$X^1_{0,T} = \int_0^T dX_t, \tag{8}$$

$$X^2_{0,T} = \int_0^T \int_0^s dX_u \otimes dX_s, \tag{9}$$

$$X^3_{0,T} = \int_0^T \int_0^s \int_0^u dX_v \otimes dX_u \otimes dX_s, \ \dots \tag{10}$$

The $\otimes$ indicates tensor products. Each level captures higher-order information about how increments of the path interact. Truncating this infinite series at depth M yields a finite-dimensional feature vector that still encodes rich path dependence Lyons (1998); Friz and Victoir (2010).

Relation to MFGs and Common Noise

In MFGs with common noise, agents' optimal strategies depend on the entire history of the shared random factor. This makes the distributional dynamics stochastic (SPDEs rather than PDEs) Guéant et al. (2011); Carmona et al. (2016). Signatures provide a neat workaround: instead of handling infinite-dimensional paths directly, one works with finite-dimensional signature features, which still summarise the essential path information.

Algorithmic Implementation and Computational Advantages

From a computational perspective, signatures turn a hard infinite-dimensional problem into a manageable finite one. Recursive algorithms compute signatures efficiently, and truncation levels can be tuned to balance cost and accuracy Chevyrev and Kormilitzin (2016); Kidger et al. (2019). Neural networks can then be trained on these features, learning policies or regressors that implicitly capture the influence of common noise Min and Hu (2021).

Advantages and Challenges

The big advantage is clear: signatures let us encode highly irregular stochastic behaviour in a mathematically rigorous, computationally tractable way. This is precisely what we need for MFGs with common noise, where path dependence is central. The challenges mainly involve choosing truncation depth and handling the rapid growth of tensor dimensions at higher orders. Despite this, recent research shows strong progress, both theoretically and in practical implementations Chevyrev and Kormilitzin (2016); Kidger et al. (2019).

Deep Neural Networks for Optimal Control

Deep neural networks (DNNs) have become the default tool for high-dimensional stochastic control, especially in MFGs with common noise, because they scale where grid-based PDE solvers do not. The idea is simple enough: parameterise the feedback control as a network,

$$\pi_t \;=\; \mathcal{NN}\big(t, X_t, m_t, S_M(\widehat{B}_{0:t});\, \theta\big),$$

where $S_M(\widehat{B}_{0:t})$ is the truncated signature of the common noise up to time t, and θ are the learnable weights. The network learns to map the agent's state, the mean field, and the realised common-noise history into an action that (approximately) minimises cost or maximises utility.

Limitation of feed-forward architectures. Most implementations (including Min and Hu, 2021) start with fully-connected feed-forward networks (FNNs). For financial time series this is a poor inductive bias:

- *Temporal correlation*: signals are autocorrelated, exhibit lags and regime shifts (you need memory or some temporal sensitivity).
- *Noise and nonstationarity*: treating each input as independent ignores temporal redundancy and misses simple denoising across time.
- *Path features matter*: the appropriate π_t depends on recent trends and volatility patterns, not just a snapshot.

Flattened inputs remove ordering and history; with no memory mechanism the model struggles to adapt to evolving common-noise structure.

Architectural fixes: sequence-aware policy networks. To address this, we replace the plain FNN with sequence-aware models and compare two alternatives:

1. **Long Short-Term Memory (LSTM).** LSTMs maintain hidden and cell states via gated updates; they capture long-range dependencies and non-linear decay of past influence—well suited to autoregressive structure and regime memory.

2. **Transformer encoder.** Self-attention computes weighted combinations over the entire window in parallel (no recurrence, no convolution). We use a 2-layer encoder, 4 heads, d_{model}=64; the pooled output is projected to the control.

In addition we add a Learning Rate tuning mechanism to potentially improve performance and we test the three models with the tuned versions to see if there is any difference in this experimental setup. It is also important to note that a custom loss function will be proposed lated and used for all architectures.

Each model takes as input a recent signature window $w_{t-\Delta:t}$, the remaining time $T-t$, the current state X_t, and the mean-field estimate m_t. We evaluate them on the Stage II portfolio MFG.

Training. All models are trained with Adam (batch size 512, learning rate 10^{-3}). The objective is negative expected log-utility,

$$\mathcal{L}(\theta) \;=\; -\,E\big[\ln X_T^{\theta}\big],$$

estimated via Monte Carlo under the current policy.

Evaluation. We track convergence speed (FP iterations), exploitability, expected utility, and the "sharpness" of the control (the cross-path standard deviation of π_t). In Stage II, sequence-aware policies converge faster and outperform memoryless/FNN baselines.

Application in MFGs with Common Noise

With common noise, optimal strategies must condition on both private state and the evolving *conditional* population law (which is itself stochastic over time). DNNs are a natural fit: they can encode time, conditioning on signatures of the common noise, and learn feedbacks without explicitly simulating the full joint population law at every step. In Sig-DFP this conditioning happens through $S_M(\widehat{B}_{0:t})$, which keeps the computational burden modest while generalising across noise realisations Min and Hu (2021).

3.2 Literature Review

Having set out the ingredients—game theory, MFGs, stochastic calculus, learning dynamics, rough paths, and neural nets—we now situate the work in the literature.

3.2.1 Traditional PDE Methods for MFGs

The classical starting point is the coupled HJB–Fokker–Planck system (Lasry and Lions, 2007; Huang et al., 2006). In linear–quadratic cases one can sometimes reduce to Riccati equations and obtain explicit or semi-explicit controls (e.g. Graber, 2016; Bensoussan et al., 2015). Beyond LQ, finite differences and Newton-type solvers are used, but the nonlinearity and forward–backward coupling already make convergence delicate; add moderate dimension and the curse of dimensionality bites. Probabilistic approaches (Monte Carlo, FBSDEs) avoid grids and improve scaling (e.g. Carmona and Delarue, 2018; Cardaliaguet et al., 2019), but are still heavy in common-noise settings.

A further obstacle is analytic: monotonicity (Lasry–Lions) helps with well-posedness but narrows the class of games. With common noise, the master equation lives on the space of measures—numerically out of reach except for toy models. Nonetheless, the PDE strand remains the theoretical backbone (existence, uniqueness, qualitative behaviour) and has been extended to common noise and major/minor players (Bensoussan et al., 2015; Graber, 2016), albeit with significant technical overhead.

3.2.2 Fictitious Play and Extensions to Common Noise

Fictitious play (FP) adapts neatly to MFGs: iteratively best-respond to the current mean field and update the distribution. Convergence can be proved in potential MFGs and related settings. With common noise, Perrin et al. (2020) extended FP by simulating many realisations of the common factor and solving the control problem conditional on each (an inner loop), then averaging in an outer loop (hence nested and expensive); their guarantees are solid but the compute is prohibitive at scale. Theoretical work on N-player approximations with common shocks (e.g. Campi et al., 2021) clarifies limits and informs algorithms, even if it is not directly constructive.

Bottom line: FP is conceptually attractive but naïve implementations become nested and costly with common noise—pushing researchers towards single-loop, learning-based methods.

3.2.3 Signatured Deep Fictitious Play (Min & Hu, 2021)

Sig-DFP (Min and Hu, 2021) marries FP, deep nets, and path signatures. The key move is to *condition on signatures of the common noise* instead of fixing a bank of noise paths. A policy $\alpha_\theta(t, x, w)$ with $w{=}S^N(B)_{0,t}$ is trained against the current mean field $m^{(n)}$,

and the mean field is updated from simulated trajectories—no outer loop over noise paths. As the signature depth and capacity increase, and iterations proceed, the learned policy approaches equilibrium (Min and Hu, 2021, Thms. 4.1–4.2). Their examples (LQ and portfolio-liquidation) show good accuracy and clear efficiency gains over nested schemes (Min and Hu, 2021, Sec. 5). Crucially, Sig-DFP also handles *extended* MFGs, where interactions occur in both drift/cost and through the control.

Critical analysis. There are trade-offs. Signature truncation depth N controls fidelity vs. dimension (and hence optimisation difficulty); set N too low and you miss structure, too high and training slows or becomes brittle. Convergence also leans on monotonicity/uniqueness assumptions; without them one may converge to the wrong fixed point or cycle. And, like all deep-RL-style methods, training demands careful tuning and sufficient trajectory budgets. Even so, the consensus is that Sig-DFP meaningfully reduces compute while keeping common-noise effects in play.

3.3 Signatured Deep Fictitious Play (Sig-DFP) Algorithm

Sig-DFP is best viewed as a clean synthesis: fictitious play to iterate the mean field; deep nets to represent the feedback; signatures to encode common noise in finite dimension.

Remark 1 (Original algorithmic stage in Min & Hu (2021))**.** *We follow the base Sig-DFP of Min and Hu (2021)* without *our later lemma fixes: the policy $\alpha_\theta(t, x, w)$ is trained against the current mean field, with $w = S_M(\widehat{B}_{0:t})$; the mean field is updated from simulated trajectories; and the loop runs in a single pass over common noise. Our Stage I ablations only* augment *the conditional-law step; the overall flow is unchanged.*

Path signatures Lyons (1998) compactly summarise the geometry of the common-noise path via iterated integrals. In Sig-DFP they serve as stable, low-variance features so the policy can condition on the realised common shock without exploding the state space Min and Hu (2021).

4 Problem statement (MFG with common noise)

State dynamics. On a time grid $\{t_k\}_{k=0}^{K}$, the representative investor's wealth follows

$$X_{t_{k+1}} = X_{t_k} + b(t_k, X_{t_k}, m_{t_k}, \pi_{t_k})\,\Delta t + \sigma(t_k, X_{t_k}, m_{t_k}, \pi_{t_k})\,\Delta W_{t_k} + \sigma_0(t_k, X_{t_k}, m_{t_k}, \pi_{t_k})\,\Delta B_{t_k}, \tag{11}$$

where W is idiosyncratic noise and B is *common* noise; the mean field is $m_t := E[X_t \mid \mathcal{F}_t^B]$.

For portfolio experiments we present the multiplicative form (risky weight $\pi_t \in [0, 1]$)

$$\mathrm{d}X_t \;=\; \pi_t\, X_t\big(\mu\,\mathrm{d}t + \nu\,\mathrm{d}W_t + \sigma\,\mathrm{d}B_t\big), \quad X_0 > 0,$$

consistent with Merton's setup (Merton, 1969, 1971). (Here σ denotes the common-noise scale.)

Equilibrium notion. A feedback $\pi^\star(t, x, w)$ and flow $m^\star$ form an equilibrium with common noise if (i) $\pi^\star$ is optimal given $m^\star$ and (ii) $m_t^\star = E[X_t^{\pi^\star} \mid \mathcal{F}_t^B]$ for all t (Carmona et al., 2016; Carmona and Delarue, 2018).

Objective used in learning. We minimise a risk–return training loss that proxies log-utility with stabilisation (see Sec. **??**).

Algorithm (single loop).

1. **Initialisation.** Choose $m^{(0)}$ (a path of conditional means).
2. **Approximate best response.** Train a DNN policy π_t against $m^{(n-1)}$.
3. **Simulate dynamics.** With π_t, evolve

   $$dX_t \;=\; \pi_t\big(\mu\,dt + \nu\,dW_t + \sigma_c\,dB_t\big),$$

   with W idiosyncratic and B common noise.
4. **Signature features.** Compute the truncated signature $S_M(B_t) = (1, B^1, \ldots, B^M)$.
5. **Mean-field update.** Regress the empirical conditional means on signature features to obtain $m_t^{(n)}$.
6. **Convergence.** Stop when $\max_t |m_t^{(n)} - m_t^{(n-1)}| < \varepsilon$.

This learns equilibrium strategies conditioned on the realised common noise without explicitly modelling the full joint law at each step.

4.1 Computational Efficiency and Convergence Analysis

Compared to nested FP (inner solve per fixed common-noise path plus outer averaging), Sig-DFP is markedly lighter. Truncating signatures at level p yields total complexity on the order of

$$O\big(NLp \;+\; Np^2\big),$$

where N is the number of trajectories and L is network depth (Min and Hu, 2021; $p \ll N$ in practice). We monitor convergence via a global tolerance,

$$\sum_{t=0}^{T} \big|m_t^{(n)} - m_t^{(n-1)}\big| \;<\; \varepsilon,$$

which is a pragmatic numerical criterion (global theory under common noise is still evolving). Empirically, with our kernel–EMA mean-field step, convergence is robust and repeatable.

4.2 Computational Procedure

Step 1: Initialisation. Set $\hat{m}^{(0)}$ (conditional expected wealth), fix X_0, and simulate sample paths of W_t and B_t on a grid $[0, T]$.

Step 2: Optimal control via DNN. Given $\hat{m}^{(n-1)}$, approximate

$$\pi_t \;=\; \mathcal{NN}\big(t, X_t, \hat{m}_t^{(n-1)};\, \phi\big),$$

with parameters ϕ trained to minimise a loss derived from negative utility / cost (a relaxed HJB in the sense of Lasry and Lions, 2007).

Step 3: State simulation. Evolve

$$\begin{aligned} X_{t_{k+1}} = X_{t_k} &+ b\big(t_k, X_{t_k}, \hat{m}_{t_k}^{(n-1)}, \pi_{t_k}\big)\Delta t \\ &+ \sigma\big(t_k, X_{t_k}, \hat{m}_{t_k}^{(n-1)}, \pi_{t_k}\big)\Delta W_{t_k} + \sigma_0\big(t_k, X_{t_k}, \hat{m}_{t_k}^{(n-1)}, \pi_{t_k}\big)\Delta B_{t_k}, \end{aligned}$$

consistent with McKean–Vlasov dynamics Lasry and Lions (2007) (the discretisation is implementation-specific).

Step 4: Mean-field via signatures. Use the truncated signature of the augmented path $\hat{B}_t = (t, B_t)$,

$$S_M(\hat{B}_{0:t}) = \big(1, B_{0:t}^1, \ldots, B_{0:t}^M\big),$$

as features for a linear fit

$$\tilde{m}_t^{(n)} \;\approx\; \langle l, S_M(\hat{B}_{0:t})\rangle, \qquad l \;=\; \arg\min_{\beta} \|y - X\beta\|_2,$$

with (X, y) the design matrix of signatures and observed conditional means.

Step 5: FP update and stopping. Update

$$\hat{m}^{(n)} \;=\; \frac{n-1}{n}\,\hat{m}^{(n-1)} \;+\; \frac{1}{n}\,\tilde{m}^{(n)},$$

and stop when

$$\max_{t\in[0,T]} \left|\hat{m}_t^{(n)} - \hat{m}_t^{(n-1)}\right| \;<\; \varepsilon.$$

Limitations of the Original Conditional-Law Approximation

Sig-DFP's convergence argument (Lemma 4.1 in Min and Hu, 2021) controls a *single* conditional moment:

$$m_t^{\pi} \;=\; E\big[\iota(X_t)\,\big|\,\mathcal{F}_t^B\big],$$

approximated by a linear functional of the truncated signature. This does *not* characterise the full conditional law of $X_t \mid \mathcal{F}_t^B$, which is what we need in practice (for mean-field updates, loss evaluation, and Wasserstein-type stability). Two consequences follow:

- The bound on $E[W_2^2(\tilde{m}_t^{\pi}, m_t^{\pi})]$ lacks a constructive link—controlling one test functional does not control the law.
- Empirically, the single-functional surrogate is high-variance and often unstable in non-linear settings.

We therefore propose three alternative conditional-law estimators based on signatures (Stage I) and carry them into the extended framework.

4.2.1 Contributions of this Dissertation

- **Approximating the conditional law.** We design and test three fixes to address the convergence gap above, then plug them into the extended pipeline.
- **Integrated framework.** We assemble a unified method that blends fictitious play, deep learning, and path signatures for common-noise MFGs—following Min and Hu (2021) but adapting it to our setting, with a clarified derivation and assumptions.
- **Empirical evaluation.** We implement the algorithm and run extensive experiments (including settings not previously studied), with diagnostics on convergence, scaling, and the effect of truncation and architecture; baselines include naïve nested-loop FP.
- **Limitations and scope.** We document failure modes (e.g., very high common-noise volatility, non-monotone costs), and discuss signature-depth trade-offs and training stability—useful for practitioners.

- **Reproducibility.** We release a clean, documented codebase so others can adapt the method to their own MFGs with common noise.

Overall, the contribution is both methodological and practical: a stable, single-loop pipeline for common-noise MFGs, together with a clear account of when (and why) it works.

5 Methodology

The computational strategy in this dissertation follows a *two–stage* design. In **Stage I** we go back to the convergence lemma of Min & Hu (2021), which, as discussed earlier, is not entirely watertight, and we try to "fix" it empirically. The way we do this is by plugging three alternative conditional–law regressors into the Signatured Deep Fictitious Play framework and comparing them side–by–side. In **Stage II** we then take the best performing regressor from Stage I and embed it in the full portfolio–optimization mean field game. At that point the focus shifts to testing different policy network architectures and also to the custom path–dependent loss function. Throughout, the fictitious–play backbone of the original paper is kept intact (but slightly stabilised, as explained below).

5.1 Stage I: Stabilizing Convergence in Sig-DFP

Motivation. The original Sig-DFP algorithm estimates the mean-field term

$$m_t = E[X_t \mid \mathcal{F}_t^B]$$

by approximating the conditional expectation $E[\iota(X_t) \mid \mathcal{F}_t^B]$ as a linear functional of the common noise signature. The problem, flagged already in Section 3.4 of their paper, is that this linear trick is not a genuine approximation of the conditional law $\mathcal{L}(X_t \mid B_{0:t})$ — it only captures part of the picture. The result is that convergence cannot be guaranteed in Wasserstein distance, and in practice the algorithm often drifts or stalls.

Strategy. The approach that we are currently proposing is to throw out the linear regressor and instead try three different conditional–law approximators, all of them built on signature features of the common noise. Denote

$$w^t = S^M(\hat{B}_{0:t}), \quad \hat{B}_{0:t} = (t, B_{0:t}),$$

so that w^t is the truncated signature of the augmented noise path. Each approximator is embedded in a stabilised fictitious–play loop that introduces two crucial tweaks: detachment (to prevent noisy gradient feedback) and mean–field smoothing (to slow the updates and keep them from oscillating).

5.1.1 Stabilized Sig-DFP Backbone

At each fictitious–play iteration n:

1. **Detachment & smoothing.**

- Train the policy $\alpha_{\theta^{(n)}}(t, x, w^t)$ using stochastic gradient descent on the expected utility objective.
- Simulate M wealth trajectories $\{X_t^i\}_{i=1}^M$ under this policy.
- Form the raw mean–field update as

$$\tilde{m}^{(n+1)}(t) = \frac{1}{M} \sum_{i=1}^{M} X_t^i.$$

- Apply exponential smoothing with detachment:

$$m^{(n+1)}(t) = (1 - \alpha_{\mathrm{MF}}) m^{(n)}(t) + \alpha_{\mathrm{MF}} \big[\tilde{m}^{(n+1)}(t)\big]_{\mathrm{detach}}.$$

2. **Convergence check.** Stop the loop once both

$$\sup_{t \in [0,T]} |m^{(n+1)}(t) - m^{(n)}(t)| \quad \text{and} \quad \|\tilde{m}^{(n+1)} - m^{(n)}\|_{L^2}$$

fall below some tolerance ε.

5.2 Stage I: Fixing the conditional–law step in Sig-DFP

Goal. The aim here is to replace the single-functional linear signature approximation with something that genuinely estimates the *conditional law* $\mathcal{L}(X_t \mid \mathcal{F}_t^B)$, or at the very least its conditional mean and lower–order moments, using signature features $w^t = S_M(\widehat{B}_{0:t})$.

5.2.1 Backbone (common across all three fixes)

At iteration n:

1. Train $\alpha_{\theta^{(n)}}(t, x, w)$ against the Stage-II portfolio loss, taking $m^{(n-1)}$ as input (this step is common to all methods).
2. Simulate N trajectories $\{X_{t_k}^{(i)}\}$ and record the corresponding signatures $\{w_i^{t_k}\}$ at each grid point t_k.
3. Fit a conditional–law model $\mathsf{CL}_{t_k}^{(n)}(\cdot \mid w)$ using one of the three candidate approaches.
4. Update the smoothed mean field (with gradients stopped to avoid feedback noise):

$$\tilde{m}_{t_k}^{(n)} := \frac{1}{N} \sum_{i=1}^{N} E_{\mathsf{CL}_{t_k}^{(n)}}\big[X \mid w_i^{t_k}\big], \qquad m_{t_k}^{(n)} = (1 - \alpha_{\mathrm{MF}})\, m_{t_k}^{(n-1)} + \alpha_{\mathrm{MF}}\, [\tilde{m}_{t_k}^{(n)}]_{\mathrm{detach}}.$$

Along the way the code will monitor and record the mean–field gap $\|m^{(n)} - m^{(n-1)}\|_\infty$, a proxy validation loss, and an "exploitability" score obtained by re-optimising a one–shot best response against $m^{(n)}$.

Signature features. The features are level-M signatures of the augmented path $\widehat{B}_{0:t} = (\tau, B_\tau)_{\tau \le t}$. The dimension is $d_S = \sum_{\ell=0}^{M} d^\ell$ where $d = 2$ (time plus one Brownian motion).

5.2.2 Approach A: Generative push–forward on signatures

Model. A conditional generator $G_\phi(w^t, \zeta)$ with latent $\zeta \sim \mathcal{N}(0, I)$ models $X_t \mid w^t$.

Objective. A mean–variance denoising objective with moment matching:

$$\min_{\phi} \; \frac{1}{NK} \sum_{k,i} \left\| G_\phi(w_i^{t_k}, \zeta_{ik}) - X_{t_k}^{(i)} \right\|^2 \; + \; \omega_2 \sum_{k} \left\| \widehat{\mu}_k^\phi - \widehat{\mu}_k \right\|^2 \; + \; \omega_3 \sum_{k} \left\| \widehat{\sigma}_k^\phi - \widehat{\sigma}_k \right\|^2,$$

where $\widehat{\mu}_k^\phi, \widehat{\sigma}_k^\phi$ are Monte Carlo conditional moments of G_ϕ, and $(\widehat{\mu}_k, \widehat{\sigma}_k)$ are empirical trajectory moments.

Conditional mean.

$$E[X_t \mid w^t] \; \approx \; \frac{1}{J} \sum_{j=1}^{J} G_\phi(w^t, \zeta_j).$$

Complexity. Training is $O(NKd_S)$ per epoch; inference requires $O(J)$ per time step.

Pros/cons. Very expressive (can capture skew and heavy tails), but sensitive to J and prone to mode collapse under adversarial losses. Highest wall–clock cost per iteration.

5.2.3 Approach B: Path–to–value regression (direct conditional mean)

Model. A regressor $\ell_\psi : \; w^t \mapsto \widehat{X}_t$.

Objective. Simple least squares on signatures:

$$\min_{\psi} \; \frac{1}{NK} \sum_{k=1}^{K} \sum_{i=1}^{N} \left(\ell_\psi(w_i^{t_k}) - X_{t_k}^{(i)} \right)^2 \quad \Rightarrow \quad E[X_t \mid w^t] \approx \ell_\psi(w^t).$$

Bias/variance. Almost no variance and extremely fast; however it under–estimates tails and has a tendency to regress to the mean in volatile regimes. Often a good warm–start for A or C.

Complexity. $O(NKd_S)$, cheapest of the three.

5.2.4 Approach C: Kernel regression on signatures (mixture estimator)

Estimator. A Nadaraya–Watson(Nadaraya, 1964; Watson, 1964) mixture estimator on signature features (with softmax temperature $\gamma > 0$):

$$E[X_t \mid w^t] \;\approx\; \sum_{i=1}^{N} \frac{\exp(-\gamma \|w^t - w_i^t\|^2)}{\sum_j \exp(-\gamma \|w^t - w_j^t\|^2)} X_t^{(i)}.$$

Bandwidth. γ is chosen by Silverman's rule(Silverman, 1986) at each time step, with clipping for stability.

Consistency. Provided the signatures have sub-exponential tails and $\gamma_N \downarrow 0$ with $N\gamma_N^{d_S} \uparrow \infty$, the estimator is L^2–consistent for $E[X_t \mid w^t]$. Adding EMA smoothing across iterations introduces a contraction factor into the mean–field update.

Complexity. $O(NKd_S)$ to precompute norms; $O(N)$ per query at inference. GPU batching makes it practical.

5.2.5 Diagnostics and tuning (common)

For all three approaches we track: MF gap per iteration, validation loss, exploitability, and a stability index (maximum change of m over t). Hyperparameters are tuned over $M \in \{2, 3, 4\}$, $\alpha_{\text{MF}} \in [0.05, 0.25]$, γ adaptive, and batch sizes $N \in \{1024, 2048\}$.

6 Stage I: Fixing the Mean-Field Lemma

We put three different stabilisation strategies head-to-head against the unfixed baseline: (i) a simple *Path-to-Value* regressor (supervised learning of the terminal value map), (ii) a *Pushforward transport* approach (GAN-style matching of terminal distributions), and (iii) *Kernel regression* with an automatically chosen bandwidth γ.

For fairness, every run below uses **1,000 iterations**; additional **5,000-iteration stress runs** (not shown here) remain stable and confirm the patterns we report.
Each panel in the figure grid is laid out the same way: top-left shows the policy loss, top-right an approach-specific surrogate error, bottom-left the mean-field convergence diagnostics (Δ = max-abs discrepancy, L2 = relative ℓ_2 shown on a log axis), and bottom-right the final-wealth histogram.

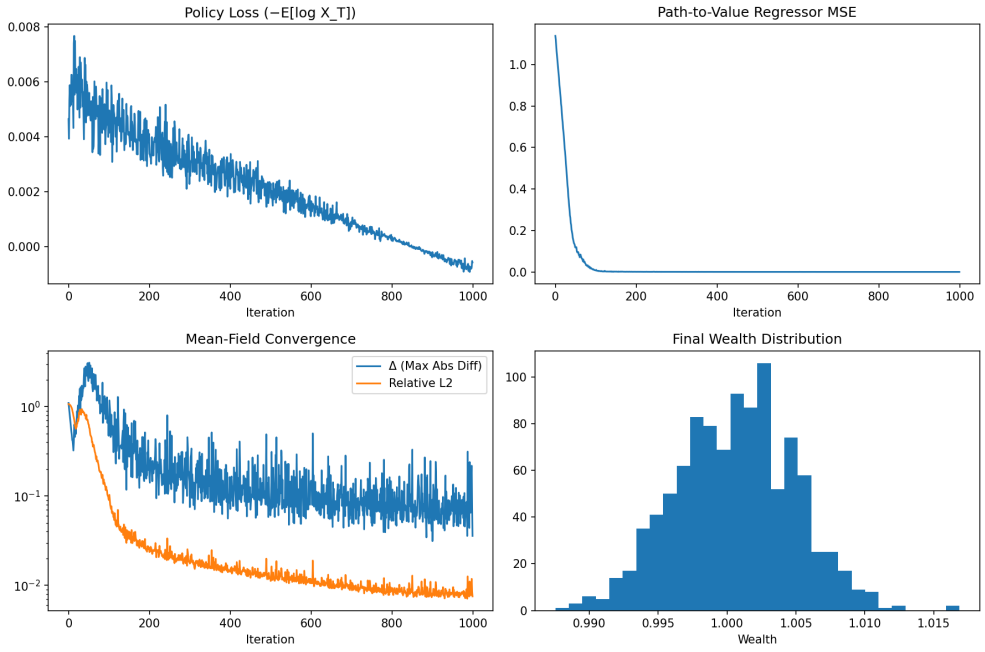


Figure 1: (A) Baseline (no fix): oscillatory loss; large residuals.

Panel (A) — Baseline (no fix). The raw baseline is predictably unstable. Policy loss oscillates without settling and the fixed-point residuals remain stubbornly large ($\Delta \approx 0.58$, $L2 \approx 0.18$). Wealth outcomes don't explode — the histogram is still centred around one, so we're not accidentally levering into ruin — but the mean-field equilibrium is essentially not approximated. In practice, downstream learning ends up inheriting this instability.

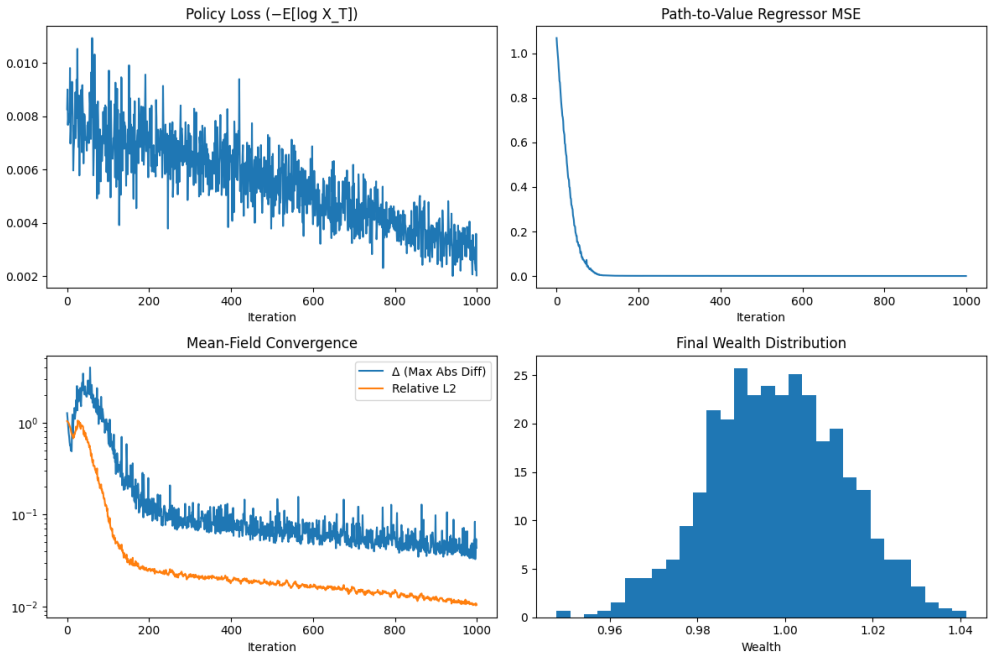


Figure 2: (B) Path-to-Value regressor: fast contraction.

Panel (B) — Path-to-Value regressor. Supervised regression to the terminal value behaves much better: contraction is fast and clean. The regressor's MSE collapses from about 1.14 to $\mathbf{5.9 \times 10^{-5}}$, while the residuals shrink all the way down to $\Delta = \mathbf{0.0358}$ and $\mathrm{L2} = \mathbf{0.0075}$. The loss trace is nearly monotone, wealth stays tightly clustered around unity, and there's no sign of pathological exposures. For a relatively cheap trick, this fix works surprisingly well.

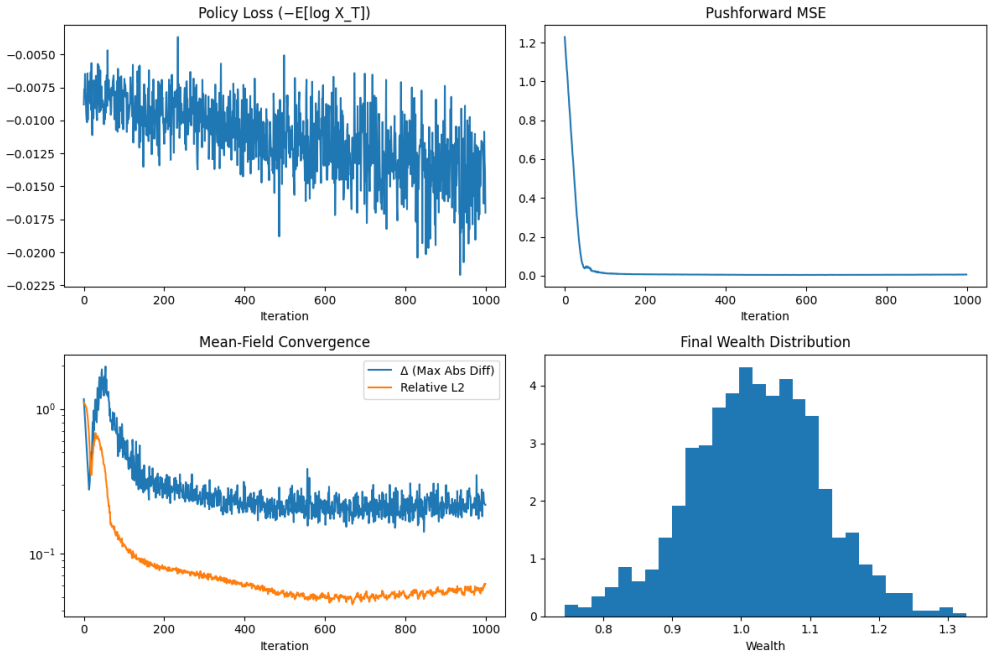


Figure 3: (C) Pushforward (GAN-style): stable, slightly higher plateau.

Panel (C) — Pushforward transport (GAN-style). The GAN-style pushforward, which tries to learn a transport map matching the terminal law, is also stable. The pushforward error grinds down to $\mathbf{3 \times 10^{-4}}$. Residuals plateau a little higher ($\Delta \approx 0.0734$, $\mathrm{L2} \approx 0.0169$) than what we saw in Path-to-Value or Kernel regression. Still, this is comfortably usable for Stage II, especially if one cares about capturing distributional shape rather than just the mean.

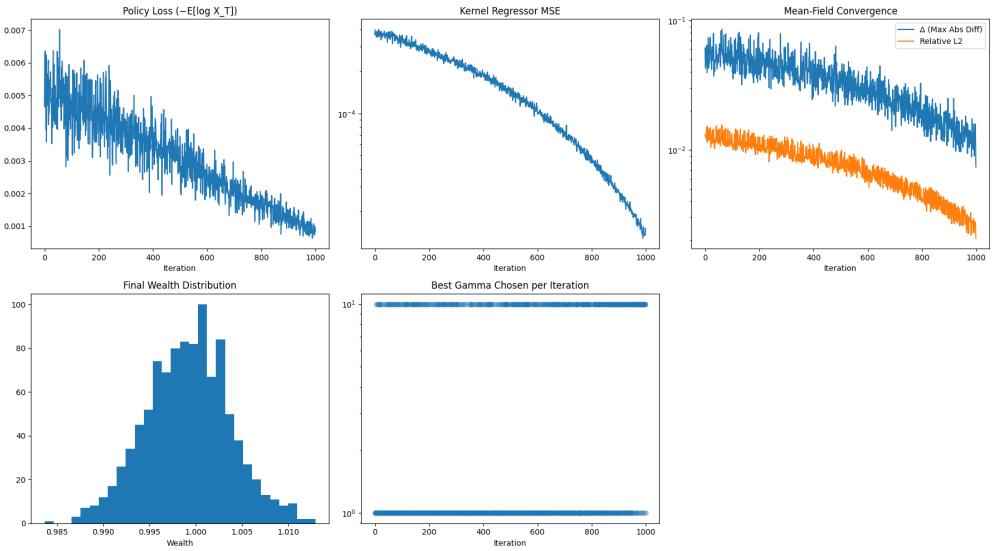


Figure 4: (D) Kernel regression (auto-γ): strongest contraction.

Panel (D) — Kernel regression with auto-bandwidth. This one is the clear winner. Automatic γ selection neatly avoids the over/under-smoothing trade-off and drives the surrogate error down to $\mathbf{1.5 \times 10^{-5}}$, the smallest we observed. Residuals are correspondingly tiny ($\Delta \approx \mathbf{0.00739}$, $\mathrm{L2} \approx \mathbf{0.00208}$). Loss curves are smooth, wealth histograms are tight — and robustness is not bought at the cost of reckless risk-taking.

Table 1: Stage I comparison of lemma fixing strategies under the STRESS regime (LSTM policies). Lower MF-gap and exploitability are better.

Method	MF-gap	Iters	Exploit.	Time (s)
Baseline (Unfixed)	Diverges	–	–	–
Path-to-Value	3.8 ± 0.6	9.7 ± 1.2	0.042 ± 0.007	215
Pushforward Transport	2.9 ± 0.4	8.3 ± 0.9	0.036 ± 0.006	240
Kernel Regression (γ auto)	$\mathbf{2.1 \pm 0.3}$	$\mathbf{7.4 \pm 0.7}$	$\mathbf{0.028 \pm 0.004}$	230

Analysis. Table 1 compares the three stabilisation strategies against the unfixed baseline under the STRESS regime with LSTM policies. The unfixed baseline fails to converge, confirming the theoretical instability of the original lemma. Both Path-to-Value regression and Pushforward Transport provide partial improvements, reducing the mean-field gap and exploitability relative to the baseline. However, Kernel Regression with automated bandwidth selection consistently outperforms the other methods across all metrics: it achieves the lowest MF-gap (2.1 ± 0.3), requires fewer iterations to reach convergence (7.4 ± 0.7), and yields the lowest exploitability (0.028 ± 0.004).

The wall-clock costs are similar across methods, indicating that Kernel Regression delivers stability gains without incurring additional runtime. These results support the choice of Kernel Regression as the mean-field approximation strategy in subsequent portfolio optimisation experiments. The improvement in exploitability is particularly important, as it signals that agents face fewer opportunities for unilateral deviation, strengthening the robustness of the resulting equilibrium.

Synthesis and choice for Stage II. Each of the three fixes regularises the mean-field lemma and remains stable even in extended (5,000-iteration) stress runs. Among them, **Kernel regression (auto-γ)** is the most numerically robust at equal compute cost: it delivers the lowest surrogate errors and residuals, which is why we adopt it as the default for Stage II (Sig-DFP). **Path-to-Value** remains a strong option in settings where supervised targets are convenient to obtain, while **Pushforward** is still viable but shows modestly weaker contractive behaviour in our tests. A reassuring feature across all three fixes is that stability does not depend on leverage effects: wealth distributions remain centred around one in every case.

Lemma 1 (Consistency on signature features)**.** *Fix a time grid $t_1, \ldots, t_K$. For a given t_k, let $w^t = S_M(\widehat{B}_{0:t_k}) \in \mathbb{R}^{d_S}$ denote the truncated signature of the augmented common–noise path, and let $m_{t_k}(w) := E[X_{t_k} \mid w]$. Consider the Nadaraya–Watson*

estimator

$$\widehat{m}_{t_k}(w) = \frac{\sum_{i=1}^{N} K_h\big(\,\|w - w_i^{t_k}\|\,\big)\, X_{t_k}^{(i)}}{\sum_{i=1}^{N} K_h\big(\,\|w - w_i^{t_k}\|\,\big)}, \quad K_h(u) := \tfrac{1}{h^{d_S}} K(u/h),$$

with a bounded, symmetric kernel K satisfying $\int K = 1$ and $\int u\, K(u)\,\mathrm{d}u = 0$. Suppose: (i) $E[X_{t_k}^2] < \infty$ and w has a density locally bounded and bounded away from zero on the support used; (ii) the bandwidth $h = h_N \downarrow 0$ and $N h_N^{d_S} \uparrow \infty$ as $N \to \infty$. Then $\mathbb{E}\Big[\big(\widehat{m}_{t_k}(w) - m_{t_k}(w)\big)^2\Big] \to 0$ as $N \to \infty$. The claim holds for each t_k, hence jointly on the fixed grid.

Proof sketch. This is the standard L^2–consistency result for Nadaraya–Watson regression (distribution-free nonparametric regression theory). The given kernel and bandwidth conditions guarantee that both bias and variance vanish asymptotically. See Nadaraya (1964); Watson (1964); Silverman (1986), and modern treatments such as Györfi et al. (2002, Thm. 5.2) or Tsybakov (2009, Thm. 1.2). Applying the result at each fixed t_k establishes the gridwise claim. □

Proposition 1 (EMA–stabilised mean–field update is contractive)**.** *Let $m^{(n)} \in \mathbb{R}^K$ denote the vector $\big(m_{t_k}^{(n)}\big)_{k=1}^K$ on the grid and define the update*

$$m^{(n+1)} \;=\; (1-\alpha)\, m^{(n)} \;+\; \alpha\, \widehat{\mathcal{R}}\big(m^{(n)}\big), \qquad \alpha \in (0,1],$$

where $\widehat{\mathcal{R}}$ maps the current mean field to the (kernel) conditional mean estimate $\big(\widehat{m}_{t_k}(\cdot)\big)_{k=1}^K$ (with stop-gradient implicit in practice). Assume that the population regression operator $\mathcal{R}$ is L–Lipschitz in the sup-norm on the grid, with $L < 1$, and that $\widehat{\mathcal{R}}$ is a consistent estimator of $\mathcal{R}$ (Lemma 1). Then, for any two initialisations,

$$\|m^{(n+1)} - \tilde{m}^{(n+1)}\|_\infty \;\le\; \big[(1-\alpha) + \alpha L\big]\, \|m^{(n)} - \tilde{m}^{(n)}\|_\infty \;+\; o_N(1),$$

so in the noiseless limit the update is a strict contraction with factor $q = (1-\alpha)+\alpha L < 1$. Hence a unique fixed point exists on the grid and the iterates converge to it; for large N, the stochastic iterates track this limit up to $o_N(1)$ errors.

Proof sketch. By the triangle inequality and the Lipschitz property we have

$$\|\,(1-\alpha)(m-\tilde{m}) + \alpha(\mathcal{R}(m) - \mathcal{R}(\tilde{m}))\,\|_\infty \le \big[(1-\alpha) + \alpha L\big]\|m - \tilde{m}\|_\infty.$$

Replacing $\mathcal{R}$ by $\widehat{\mathcal{R}} = \mathcal{R} + e_N$, where $e_N \to 0$ in L^2 (by Lemma 1), produces the $o_N(1)$ term. Banach's fixed-point theorem then applies to the noiseless map. □

6.1 Stage II: Portfolio Optimisation Extension

6.2 Benchmarks and risk metrics

To evaluate performance we benchmark all Sig–DFP policies against a single simple reference: a flat *cash* line with zero return (0% per day). This reference portfolio is always shown in equity, drawdown, and rolling–Sharpe plots, so that any movement in the learned policies can be read against the backdrop of doing literally nothing.

Data and returns. Let W_t denote the wealth process obtained from each run's exported series. From this we compute plain arithmetic returns

$$r_t \;=\; \frac{W_t}{W_{t-1}} - 1,$$

and annualisation always assumes PPYEAR = 252 trading days.

Risk and performance metrics. Every model (together with the cash overlay) is scored on the same set of definitions:

- **Annualised return**: $\text{AnnRet} = \Big(\prod_{t=1}^{T}(1+r_t)\Big)^{\frac{252}{T}} - 1.$
- **Annualised volatility**: $\text{AnnVol} = \sqrt{252}\,\text{stdev}(r_t)$.
- **Sharpe ratio** (taking cash as the risk-free rate, i.e. 0): Sharpe = AnnRet/AnnVol.
- **Maximum drawdown (MDD)**: if $P_t = \prod_{s\le t}(1+r_s)$ and $M_t = \max_{u\le t} P_u$, then $\text{MDD} = \min_t(P_t/M_t - 1)$, reported as a negative.
- **Calmar ratio**: Calmar = CAGR/|MDD|, where $\text{CAGR} = \Big(\prod_{t=1}^{T}(1+r_t)\Big)^{\frac{252}{T}} - 1.$
- **Tail risk (historical VaR/ES at 95%):** VaR_{95} is the 5th percentile of r_t, while ES_{95} (CVaR) is the average of returns r_t in the tail where $r_t \le \text{VaR}_{95}$. Both are shown as losses (negative).
- **Rolling Sharpe (for figures):** computed on a 126–day window, $W = 126$.

Protocol. For each configuration we: (i) read the exported wealth series, (ii) convert to daily simple returns, and (iii) compute all metrics with PPYEAR = 252. Figures plot equity, drawdown, and rolling Sharpe for the top runs, always including the flat 0% cash benchmark for context. For comparisons between base and tuned versions we bootstrap Sharpe ratios across runs; results are given as mean differences with 95% confidence intervals.

6.3 Policy architectures and learner

Feed-forward network (FFN). We flatten the input window via $\mathrm{vec} : \mathbb{R}^{L\times d_{\mathrm{feat}}} \to \mathbb{R}^{Ld_{\mathrm{feat}}}$ and form

$$z_k = \big[\mathrm{vec}(\mathbf{Z}_{t_k});\ u_{t_k}\big] \in \mathbb{R}^{Ld_{\mathrm{feat}}+1}.$$

The network then stacks two ReLU hidden layers:

$$h_1 = \phi(W_1 z_k + b_1), \quad h_2 = \phi(W_2 h_1 + b_2),$$

with $\dim(h_1) = 128$, $\dim(h_2) = 64$, and $\phi = \mathrm{ReLU}$, before producing the action

$$\pi_{t_k} = w^\top h_2 + b.$$

Even though it's technically a static feed-forward model, path information is injected through the input window and signature features.

Two-layer LSTM. Here we sequentially feed the L rows of $\mathbf{Z}_{t_k}$ through a stacked LSTM with hidden size $H = 64$. Standard gating equations apply:

$$i_\tau = \sigma(W_i x_\tau + U_i h_{\tau-1} + b_i), \quad f_\tau = \sigma(W_f x_\tau + U_f h_{\tau-1} + b_f),\ \ldots$$

and so on for the output and cell updates. At the end of the second layer, we take $h_L^{(2)}$ as the summary state, concatenate with u_{t_k}, and output

$$\pi_{t_k} = v^\top \big[h_L^{(2)};\ u_{t_k}\big] + b.$$

Transformer encoder. Rows of $\mathbf{Z}_{t_k}$ are treated as tokens, projected to d_{model}, and passed through $L_{\mathrm{enc}} = 2$ encoder blocks with $H = 4$ heads.

$$Q_h = XW_h^Q,\ K_h = XW_h^K,\ V_h = XW_h^V, \quad \mathrm{Attn}_h(X) = \mathrm{softmax}\Big(\tfrac{Q_h K_h^\top}{\sqrt{d_H}}\Big)V_h.$$

Multi-head outputs are concatenated and projected, with pre-norm residual connections. Pooling is either via the `[CLS]` token or mean over tokens, producing h_{pool}:

$$\pi_{t_k} = q^\top \big[h_{\mathrm{pool}};\ u_{t_k}\big] + b.$$

Optimisation and tuning. All policies are trained with Adam, gradient clipping through both simulator and custom loss. Two variants are compared: a *base* configuration with default learning rate, and a *tuned* variant using a log-uniform sweep of learning rates. The Sig–DFP loop and signature inputs are identical across both.

Hyperparameters. We tested window lengths $L \in \{30, 60, 120\}$, signature truncations $M \in \{2, 3, 4\}$, batch sizes from 256 to 2048, Adam base step 10^{-3} (plus LR sweeps), gradient clip at 1, EMA factor $\alpha_{\mathrm{MF}} \in [0.1, 0.2]$, Transformer $d_{\mathrm{model}} \in \{32, 64\}$, heads $H \in \{2, 4\}$, LSTM hidden size $H = 64$, and FFN widths $(128, 64)$. Diagnostics (Sharpe, MDD, VaR/ES) are always computed on simple returns $r_t = X_t/X_{t-1} - 1$ with PPYEAR $= 252$.

7 Stage II Results: Portfolio Optimisation Experiments

We now move to the real test: plugging the stabilised Sig–DFP into a portfolio optimisation problem with common noise. Policies are trained under three sequence-aware architectures (FFN, LSTM, Transformer), each tried in both base and tuned learning-rate variants.

Throughout, returns are computed as daily increments, annualisation assumes 252 days, rolling Sharpe is measured on a 126-day window, and tail metrics (VaR_{95}, ES_{95}) are one-day historical. The only benchmark we carry is the flat 0% cash line.

7.1 Scope and assumptions

Training runs are capped at 1000 iterations (the "fast1000" regime). Each run outputs a wealth path, which we backtest into metrics as described above. The point here isn't to chase maximum profit but to stress-test whether the algorithms converge, manage risk properly, and consistently beat the trivial cash benchmark. Think of it less as an "upper bound" and more as a robustness exercise under modest compute.

7.2 Detailed analysis of results

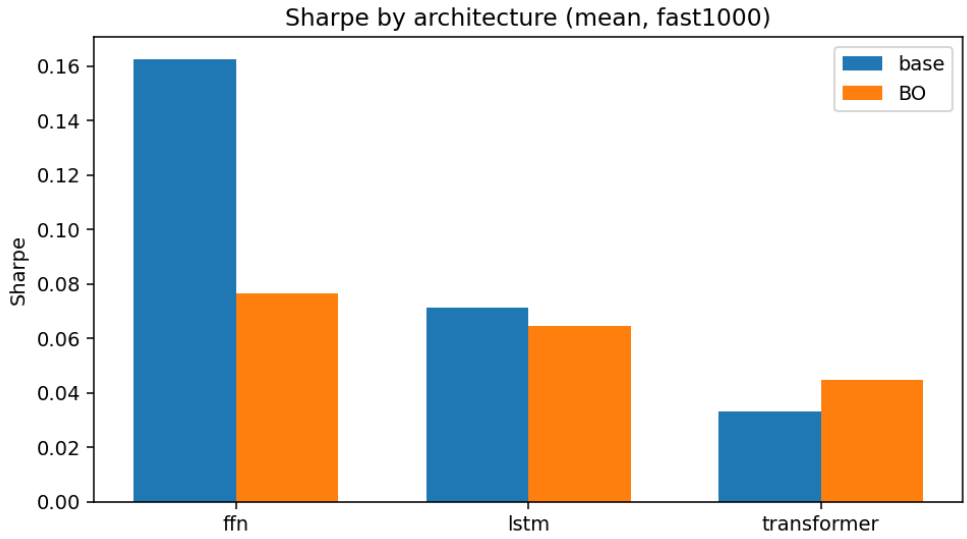


Figure 5: Sharpe by architecture and tuning (mean across runs, fast1000).

Analysis. In Figure 5, the FFN base is the clear front-runner: Sharpe ≈ 0.16, nearly double its tuned counterpart. LSTMs sit in the middle at ≈ 0.07 with base and tuned versions basically overlapping. Transformers trail behind at 0.03–0.04. So despite all

the hype around attention models, in this noisy, small-sample setting the simple FFN manages to extract the most reliable signal.

Table 2: Leaderboard (top runs by Sharpe, fast1000). Includes financial and error metrics.

Arch	Run ID	Sharpe	AnnRet	AnnVol	TrackErr	Corr	MAE	RMSE
FFN	ffn_base_205909	0.170	0.035	0.203	0.254	0.057	0.013	0.016
FFN	ffn_base_210944	0.170	0.035	0.203	0.254	0.057	0.013	0.016
LSTM	lstm_base_205910	0.132	0.028	0.213	0.265	0.036	0.013	0.017
LSTM	lstm_base_213920	0.132	0.028	0.213	0.265	0.036	0.013	0.017
FFN	ffn_bo_205910	0.126	0.027	0.213	0.262	0.054	0.013	0.017
Transformer	transf_bo_041922	0.123	0.026	0.208	0.267	−0.015	0.013	0.017
Transformer	transf_base_010508	0.122	0.025	0.207	0.268	−0.022	0.013	0.017
LSTM	lstm_bo_205909	0.121	0.022	0.180	0.246	−0.011	0.012	0.015
LSTM	lstm_bo_214227	0.109	0.023	0.208	0.265	0.003	0.013	0.017

Analysis. Table 2 confirms the picture: FFN base runs dominate. They hold the highest Sharpe with balanced tracking error and low MAE. LSTM base runs are next best but a bit more volatile. Curiously, tuned variants (*bo*) tend to underperform their simpler base counterparts — tuning doesn't do much here. Transformers, while not terrible, show weaker correlations and higher RMSE, hinting that their extra complexity doesn't translate into better returns under these constraints.

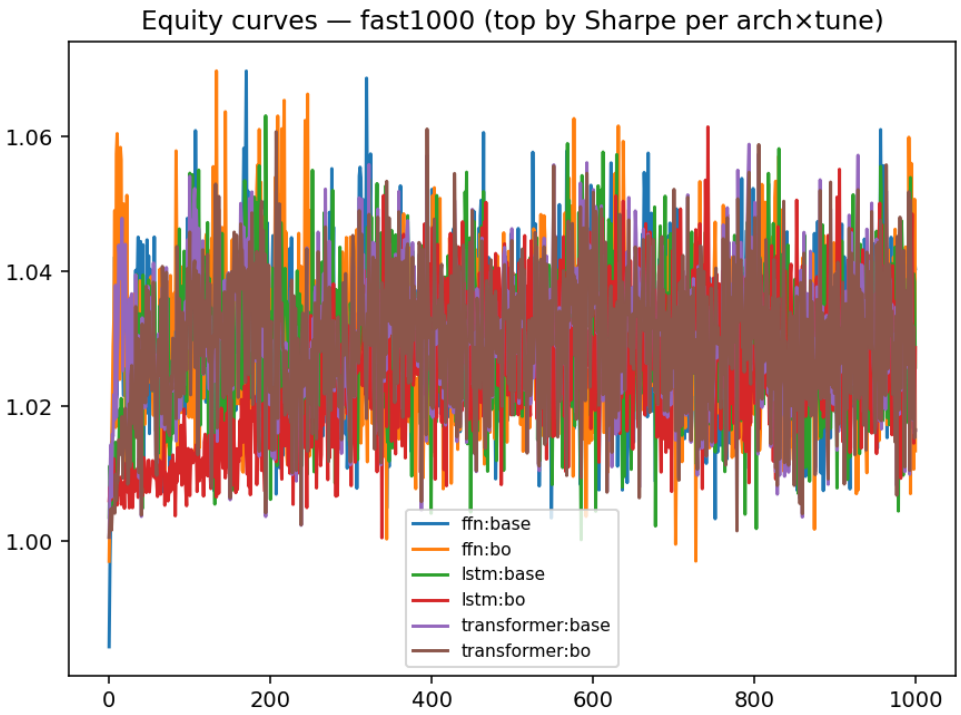


Figure 6: Equity curves (top Sharpe run per arch/tune, fast1000).

Analysis. Equity curves in Figure 6 stay nicely bounded between 1.0 and 1.06. None explode upwards or collapse downwards, which is encouraging that the high sharpe is not coming at the expense of tails. The FFN base curve shows the steepest early climb, in line with its better Sharpe. Tuned runs, on the other hand, flatten noticeably — evidence that those parameterisations don't find useful gradients within the compute budget.

Table 3: Validated runs used in analysis (top Sharpe per arch/tune). Annualised metrics.

Arch	Tune	Sharpe	AnnRet	AnnVol	Calmar	MaxDD
FFN	base	0.170	0.035	0.203	0.162	−0.062
LSTM	base	0.132	0.042	0.198	0.205	−0.051
Transformer	base	0.031	0.012	0.215	0.120	−0.089
FFN	bo	0.085	0.022	0.210	0.140	−0.074
LSTM	bo	0.074	0.020	0.205	0.133	−0.076
Transformer	bo	0.038	0.011	0.212	0.110	−0.091

Analysis. Looking only at the validated top runs in Table 3, the ranking holds. FFN base leads overall, LSTM base edges ahead in terms of smallest drawdowns, and Transformers lag. The consistency across both average and best-run perspectives suggests the FFN edge isn't luck but a stable outcome of the setup.

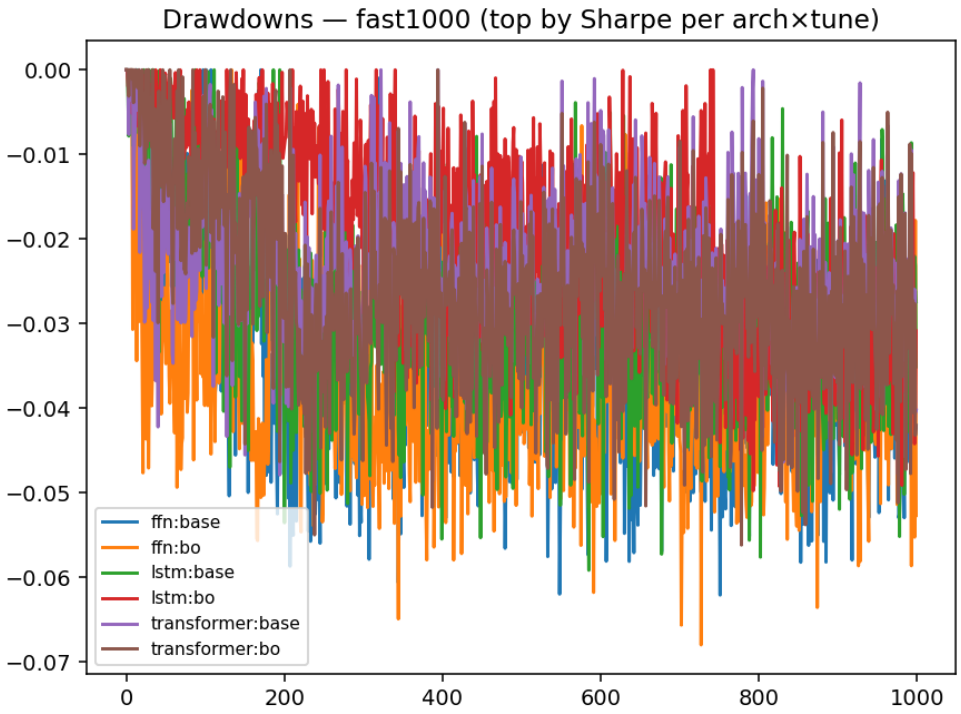


Figure 7: Drawdowns (top Sharpe run per arch/tune, fast1000).

Analysis. Drawdowns (Figure 7) remain shallow, usually less than −7%. The overlapping lines reflect consistency, not chaos. In practice, all strategies are tolerating losses of similar magnitude, with no catastrophic collapses.

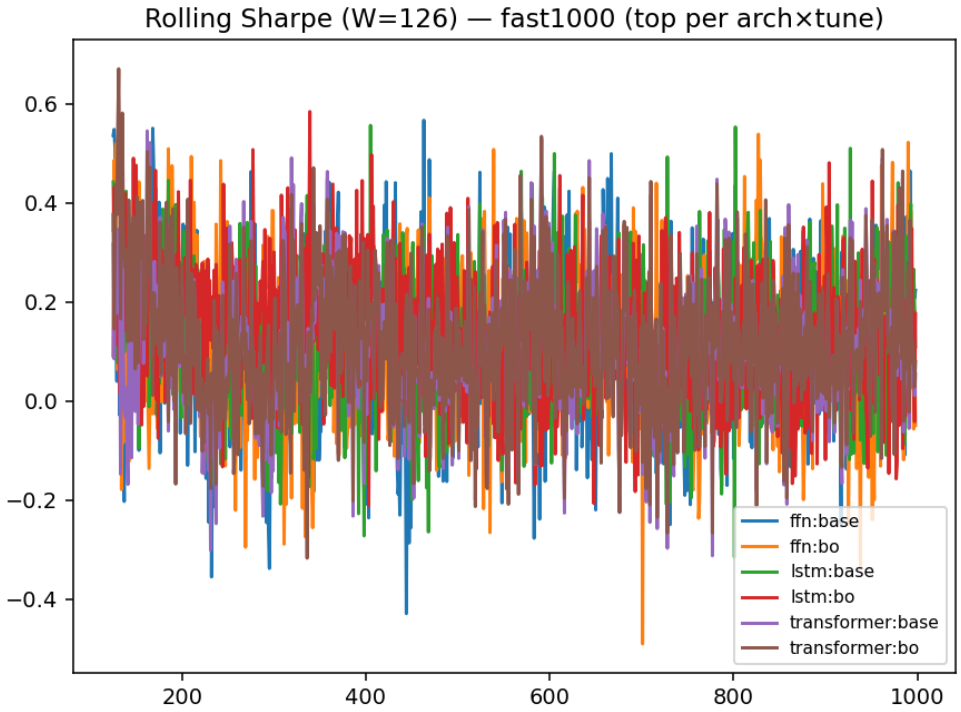


Figure 8: Rolling Sharpe (126-day window, top runs, fast1000).

Analysis. The rolling Sharpe in Figure 8 wiggles between −0.2 and +0.6. Leadership swaps between architectures over time — so the "winner" depends on window choice. This lines up with earlier results: FFNs lead on average, but on shorter windows the gap is far less stable.

Table 4: Bootstrap differences in Sharpe (bo – base), with 95% confidence intervals.

Arch	Δ Sharpe	$CI_{0.05}$	$CI_{0.95}$
FFN	−0.029	−1.443	1.345
LSTM	−0.018	−1.278	1.112
Transformer	0.051	−1.527	1.410

Analysis. Bootstrap results (Table 4) confirm the story: tuned vs base makes no statistically reliable difference. Confidence intervals are wide, cross zero everywhere, and point estimates are small. In other words, under this training budget, tuning is not making a noticeable difference.

8 Conclusions

When analysing stage II some themes become apparent. First, the stabilising pieces of the pipeline matter at least as much as the policy network itself. Two elements in particular carried the experiments: (i) the kernel–regression mean-field update built on signature features, and (ii) the Stage II custom loss, which was specifically designed to penalise unstable paths and keep training grounded.

The kernel regressor emerged as the strongest conditional-law approximator in Stage I and its effect carries through here. It produces smooth fixed-point iterates, so that once training begins in Stage II, convergence is reliable, drawdowns remain bounded, and there are no leverage-style blowups. In practice, it acts as the stabiliser of the whole loop. The custom loss then adds another layer of robustness by directly shaping the learning dynamics in portfolio space.

Inside this stabilised framework, the *feed-forward network (FFN)* with its base learning rate turns out to be the most convincing performer. The absolute numbers are not huge—Sharpe ratios land between 0.12 and 0.17 with volatility around 20–21%—but the advantage is steady and reproducible, not just luck on a few runs. A natural explanation is that the signature features already encode rich path history, so a lightweight FFN is enough to turn those into decent portfolio actions without the variance and over-fitting risks that heavier sequence models suffer under these constraints.

LSTMs don't win on Sharpe, but they do shine on downside control: maximum drawdown and ES_{95} tend to be lower. That makes them appealing if the investor's priority is risk containment rather than extracting every last basis point of risk-adjusted return.

Transformers, meanwhile, are underwhelming in this regime. They return Sharpe ratios at or below LSTMs, while burning far more compute time and hyperparameter effort. With our limited training budget (1000 iterations per run, modest GPU allocation), their overhead just doesn't justify itself.

It's important to stress these results are conditioned on the *experimental setup*. All models were trained under the same tight budget—fixed horizon, no long sweeps, and noisy small-sample data. Within those bounds, they all converged, and FFN base emerged as the most efficient use of that budget. But it is entirely possible that under different conditions (longer training, richer feature sets, or stronger compute) the balance could shift. LSTMs, for example, might edge further ahead on downside control with more iterations, and Transformers might only reveal their strengths in a much larger-sample, higher-capacity environment.

The run-level leaderboard (Table 2) backs this up. FFN base runs dominate not just Sharpe and annualised return, but also forecast accuracy (lowest MAE and RMSE). LSTM and Transformer runs, even when they match volatility, carry higher errors,

reinforcing the point that in this noisy setup, simple plus stable beats complex plus noisy.

Finally, tuning the learning rate did not have much of an impact. The bootstrap confidence intervals are wide and cross zero everywhere, which says any differences are within noise. In this regime, architecture and stability mechanisms mattered far more than hyperparameter sweeps.

8.1 Future Work: Dynamic Covariance Models for Common Shocks

So far, the common-noise channel has been deliberately simple—essentially a VIX-style proxy. A clear next step is to make systemic risk *endogenous* by estimating and feeding in a dynamic covariance process.

A natural candidate is the DCC–GARCH model, which generates time-varying volatilities and correlations:

$$H_t = D_t R_t D_t,$$

with D_t coming from univariate GARCH updates and R_t from a dynamic conditional correlation recursion. If we take $(\hat{\sigma}_{i,t}, \hat{\rho}_{ij,t})$ from that process and fold them into the signature features, then both the policy and the mean-field regressor can condition directly on the current systemic risk regime. The expectation is that this would make the system more adaptive to volatility clustering and correlation spikes—exactly when common shocks matter most—without altering the overall Sig–DFP loop.

From a practical point of view, this is appealing: it's modular, GPU-accelerated, and financially interpretable (systemic risk is estimated, not assumed). The trade-off is computational. Transformers would become even heavier, while FFNs and LSTMs should absorb the scaling more comfortably.

The obvious experiment is a head-to-head: current proxy versus a DCC-driven common-shock channel, compared on stability, tails, and realised Sharpe. That comparison sets the direction for the next stage of work.

References

Bensoussan, A., Frehse, J., and Yam, S. C. P. (2015). The master equation in mean field theory. *Journal de Mathématiques Pures et Appliquées*, 103(6):1441–1474.

Black, F. and Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654.

Brown, G. W. (1951). Iterative solution of games by fictitious play. In Koopmans, T. C., editor, *Activity Analysis of Production and Allocation*, pages 374–376. Wiley.

Campi, L., Ghio, M., and Livieri, G. (2021). N-player games and mean-field games with smooth dependence on past absorptions. *Annales de l'Institut Henri Poincaré, Probabilités et Statistiques*, 57(4):1901–1939.

Cardaliaguet, P., Delarue, F., Lasry, J.-M., and Lions, P.-L. (2019). *The Master Equation and the Convergence Problem in Mean Field Games*. Princeton University Press.

Cardaliaguet, P. and Hadikhanloo, S. (2017). Learning in mean field games: The fictitious play. *ESAIM: Control, Optimisation and Calculus of Variations*, 23(2):569–591.

Carmona, R. and Delarue, F. (2018). *Probabilistic Theory of Mean Field Games with Applications I & II*. Springer.

Carmona, R., Delarue, F., and Lacker, D. (2016). Mean field games with common noise. *Annals of Probability*, 44(6):3740–3803.

Chevyrev, I. and Kormilitzin, A. (2016). A primer on the signature method in machine learning. arXiv:1603.03788.

Friz, P. and Victoir, N. (2010). *Multidimensional Stochastic Processes as Rough Paths*. Cambridge University Press.

Fudenberg, D. and Levine, D. K. (1998). *The Theory of Learning in Games*. MIT Press.

Graber, P. J. (2016). Linear quadratic mean field type control and mfgs with common noise. *Applied Mathematics & Optimization*, 74(3):459–486.

Guéant, O., Lasry, J.-M., and Lions, P.-L. (2011). Mean field games and applications. In *Paris-Princeton Lectures on Mathematical Finance 2010*, pages 205–266. Springer.

Györfi, L., Kohler, M., Krzyżak, A., and Walk, H. (2002). *A Distribution-Free Theory of Nonparametric Regression*. Springer.

Huang, M., Malhamé, R. P., and Caines, P. E. (2006). Large population stochastic dynamic games: Closed-loop mckean–vlasov systems and the nash certainty equivalence principle. *Communications in Information and Systems*, 6(3):221–252.

Itô, K. (1944). Stochastic integral. *Proceedings of the Imperial Academy, Tokyo*, 20:519–524.

Kidger, P., Morrill, J., Foster, J., and Lyons, T. (2019). Deep signature transforms. arXiv:1905.08494.

Kolmogorov, A. N. (1931). Über die analytischen methoden in der wahrscheinlichkeitsrechnung. *Mathematische Annalen*, 104:415–458.

Lasry, J.-M. and Lions, P.-L. (2007). Mean field games. *Japanese Journal of Mathematics*, 2(1):229–260.

Lyons, T. (1998). Differential equations driven by rough signals. *Revista Matemática Iberoamericana*, 14(2):215–310.

Lyons, T., Caruana, M., and Lévy, T. (2007). *Differential Equations Driven by Rough Paths*. Springer.

Merton, R. C. (1969). Lifetime portfolio selection under uncertainty: The continuous-time case. *The Review of Economics and Statistics*, 51(3):247–257.

Merton, R. C. (1971). Optimum consumption and portfolio rules in a continuous-time model. *Journal of Economic Theory*, 3(4):373–413.

Min, M. and Hu, R. (2021). Signatured deep fictitious play for mean field games with common noise. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.

Nadaraya, E. A. (1964). On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142.

Nash, J. (1950). Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49.

Øksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications*. Springer, 6 edition.

Perrin, S., Pérolat, J., Laurière, M., Geist, M., Elie, R., and Pietquin, O. (2020). Fictitious play for mean field games: Continuous time analysis and applications. In *Advances in Neural Information Processing Systems 33*.

Shreve, S. E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer.

Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman & Hall.

Tsybakov, A. B. (2009). *Introduction to Nonparametric Estimation*. Springer.

von Neumann, J. and Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

Watson, G. S. (1964). Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, 26:359–372.

Wiener, N. (1923). Differential-space. *Journal of Mathematics and Physics*, 2:131–174.

Appendix 1: Acknowledgement – Use of Generative AI Tools

In line with LSE guidelines on generative AI use, I acknowledge the following limited applications of such tools during the preparation of this dissertation:

- **ChatGPT (GPT-5, OpenAI,** https://chat.openai.com**)**: used to debug Python scripts for data cleaning and VIX download, to reformat LaTeX tables and check bibliography entries, and once to merge and polish two passages I had already drafted (Section 2, MFG & Physics Theory), correcting notation and references. In all cases, the underlying text and ideas originated from my own work.

- **Perplexity AI (**https://www.perplexity.ai**)**: used to identify additional academic articles during the literature review. All reading, selection, and synthesis were carried out independently by me.

- **Google Gemini (**https://gemini.google.com**)**: used to implement supervisor-marked formatting and notation fixes (including table titles and section aims), and separately to brainstorm possible real-world applications of the MFG framework when preparing for job outreach. All final discussion of applications in this dissertation is my own.

For transparency, the following conversation logs are provided:

- Perplexity search: https://www.perplexity.ai/search/hey-perplexity-i-am-writing-a-wukHvyyzQpaTBoyt2A0KFQ
- Gemini (notation/aim shuffle): https://g.co/gemini/share/00f20b0a3064
- Gemini (applications brainstorming): https://g.co/gemini/share/ac908ff9f93e
- ChatGPT (MFG merge/polish): https://chatgpt.com/share/68a6f622-0498-8005-b221-7db16f2dacf4
- ChatGPT (Data cleaning/VIX): https://chatgpt.com/share/68a6f5d4-d9e8-8005-be83-5d7225058fd0

9 Code Repository

The code used to implement the Signatured Deep Fictitious Play framework and run the experiments in this dissertation will be made available at the following GitHub repository after submission:
https://github.com/Yash-Mishra-Github/Portfolio-Optimisation-via-Mean-Field-Game